CS3002E Compiler Design

Monsoon 2025

Lecture 29, 30

Translation of Function Calls and Array References

Department of Computer Science & Engineering
NIT Calicut

Function calls and returns: 3-address instructions

- Parameter: `param` x
- Function call: y=`call` p, n where integer n indicates the number of actual parameters.
- Return: `return` y, where y represents a returned value

Function calls

$y = f\ (x_1,\ x_2\ \ldots x_n)$ is translated to the following sequence of 3-address instructions:

```
param x_1
param x_2
...
param x_n
t = call f, n
y = t
```

Function call: Grammar

$$
\begin{aligned}
E &\rightarrow id\ (\ A\ ) \\
A &\rightarrow \epsilon \mid E,\ A
\end{aligned}
$$

Function call: Translation

A call **id** ($E_1, \; E_2, \ldots \; E_n$) generates:

- instructions for evaluating the parameters $E_1, \; E_2, \ldots \; E_n$ to addresses.
- a sequence of `param` instructions - one for each parameter.
- a 3-address call instruction.

Function call: Example

The statement `x = f(a+b)` is translated to:

$\mathtt{t}_1$ `= a+b`
`param` $\mathtt{t}_1$
$\mathtt{t}_2$ `= call f, 1`
`x =` $\mathtt{t}_2$

3-address Indexed copy instructions

$x = y[i]$: sets x to the value in the location i memory units beyond location y.

$x[i] = y$: sets the content of the location i units beyond x to the value in y.

Addressing Array Elements

- indexed $0, \ 1, \ \ldots n-1$
- elements stored at consecutive locations
- *base* is the relative address of the storage allocated for the array
- *row-major* order [1] - elements in first row followed by elements in second row and so on

[1] *column-major* order stores elements column-by-column

Addressing Array Elements: One-dimensional array:

If the width of each element in array A is w,

relative address of $A[i]$ is $base + i \times w$

Addressing Array Elements: One-dimensional array:

Suppose the width of each element in the array a is 4, the expression a[i] in the source code is to be translated as:

t_1 = i*4

t_2 = a[t_1]

Addressing Array Elements: Two-dimensional array

Let w_1 be the width of a row and w_2 be the width of an element in array A.

relative address of $A[i_1][i_2]$ is $base + i_1 \times w_1 + i_2 \times w_2$

Expression grammar modified to include Array References

$$E \rightarrow E + E \mid id \mid L$$

$$L \rightarrow id\ [E] \mid L\ [E]$$

Translation of Array References

$$E \rightarrow L \qquad \{E.addr = newTemp();\ gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']'\ );\}$$

$$L \rightarrow id\ [E] \qquad \{L.array = id.entry;\ L.type = L.array.type.elemtype;\ L.addr = newTemp();\ gen(L.addr\ '='\ E.addr * L.type.width);\}$$

L.array - pointer to the Symbol Table entry for the array name.

L.type - type of the subarray generated by *L*

L.array.base - base address of the array

L.addr - temporary that holds the offset for the array reference generated by *L*

t.elemtype, for an array type *t*, gives the element type of *t*

Translation of Array References

$$
\begin{aligned}
L \rightarrow L_1\ [E] \qquad & \{L.array = L_1.array;\\
& L.type = L_1.type.elemtype;\\
& t = newTemp();\\
& L.addr = newTemp();\\
& gen(t\ '=' \ E.addr\ '*'\ \ L.type.width);\\
& gen(L.addr\ '=' \ L_1.addr\ '+'\ \ t);\}
\end{aligned}
$$

Translation of Array References

$$S \rightarrow L = E \qquad \{gen(L.array.base\ '['\ L.addr\ ']'\ '='\ E.addr);$$

generates an indexed copy instruction to assign the value denoted by E to the location denoted by L

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Section 6.4